Cross Platform Development With Xamarin

Er. Piyush Pant

Xamarin

- Xamarin is a Microsoft-owned open-source framework that enables developers to build cross-platform mobile apps using a single C# codebase, while still delivering native performance and user experience.
- It is widely used for creating apps that run on iOS, Android, and Windows without maintaining separate codebases.

Overview of Xamarin

Language & Framework: Built on **.NET and C#**, Xamarin allows developers to write code once and reuse it across multiple platforms.

Native Access: Provides **bindings to native APIs** of iOS and Android, ensuring apps can access device features like notifications, graphics, and sensors.

Open Source: Xamarin SDK is fully open-source under the **MIT license**, encouraging community contributions and innovation

Key Features

Xamarin.Forms: Enables developers to create a **shared UI layer** that renders natively across platforms, ideal for apps with similar interfaces.

Xamarin.iOS & Xamarin.Android: Allow platform-specific customization while still sharing core business logic.

Code Sharing: Up to **90% of code can be reused** across platforms, reducing development time and costs.

Integration with Visual Studio: Provides powerful debugging, testing, and deployment tools.

Performance: Unlike older cross-platform frameworks (e.g., Cordova, Titanium), Xamarin ensures **native-like performance** and user experience.

Cross Platform Development

- Cross-platform development allows developers to build applications that run on multiple operating systems (like Android, iOS, and Windows) using a single codebase, saving time and resources while ensuring consistent user experiences.
- Popular frameworks include Xamarin, Flutter, and React Native, each offering different strengths in performance, UI flexibility, and ecosystem support.

What is Cross-Platform Development?

- **Definition**: The practice of creating software applications that work across multiple platforms using shared code.
- **Goal**: Reduce duplication of effort while maintaining quality and performance.
- **Platforms Covered**: Primarily **Android and iOS**, but some frameworks also support Windows, macOS, and web apps.

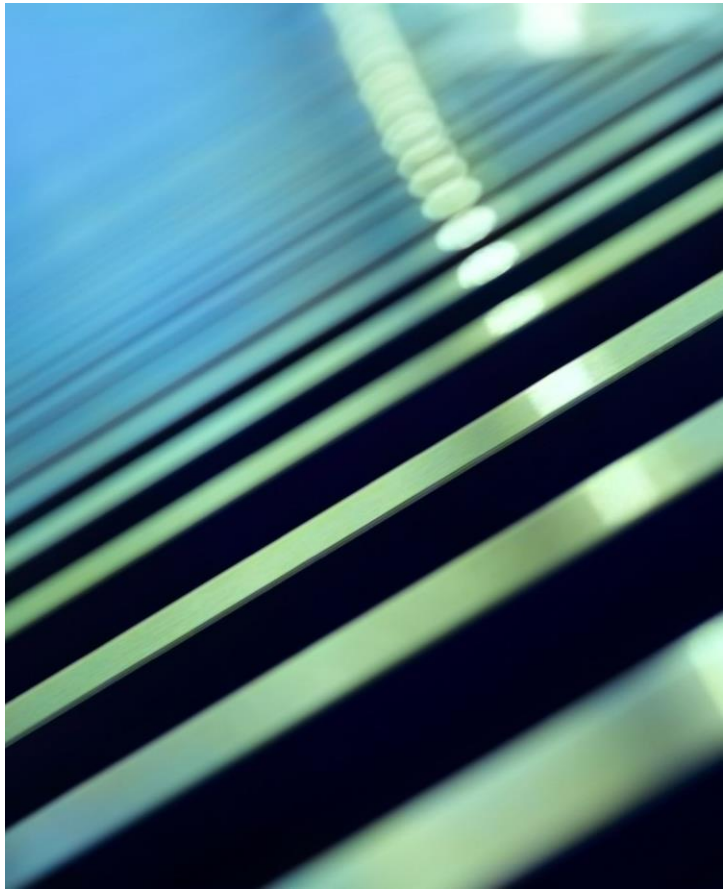
Benefits

Efficiency: Write once, deploy everywhere — up to **70–90% code reuse** depending on framework.

Cost Savings: Lower development and maintenance costs compared to separate native apps.

Consistency: Uniform business logic and user experience across devices.

Faster Updates: Easier to roll out bug fixes and new features simultaneously.



Aspect	Native Development	Cross-Platform Development
Codebase	Separate for each OS	Shared across platforms
Performance	Best possible, fully optimized	Near-native, may have minor overhead
Development Speed	Slower, more resource-intensive	Faster, cost-effective
UI/UX	Fully tailored to each platform	Consistent but sometimes less flexible
Maintenance	Multiple updates required	Single update for all platforms

Leading Frameworks (2025–2026)

Xamarin / .NET MAUI: Microsoft-backed, strong for enterprise and .NET developers. Offers native API access and integration with Visual Studio.

Flutter (Google): Uses Dart language, excels at custom UI rendering and smooth animations.

React Native (Meta): JavaScript-based, large ecosystem, widely adopted for consumer apps.

Summary

- Cross-platform development is the go-to solution for businesses that want to reach both Android and iOS users quickly and cost-effectively.
- Xamarin is ideal for enterprises already invested in .NET.
- Flutter shines in UI-heavy apps.
- React Native is strong for consumer-facing apps with large ecosystems.

XAML

XML (Extensible Markup Language)

- A **general-purpose markup language** used to store and transport data.
- Focuses on **data representation** (like `<Student><Name>Piyush</Name></Student>`).
- Doesn't know anything about UI or how data should look.

XAML (Extensible Application Markup Language)

- A **specialized form of XML** created by Microsoft.
- Used to **design UI** in technologies like **WPF, UWP, and Xamarin.Forms**.
- Focuses on **layout and controls** (like `<Button Text="Click Me" />`).
- Supports **data binding, events, and resources**, which XML doesn't.

- **XML = describes data**
- **XAML = describes UI (using XML syntax)**

Example

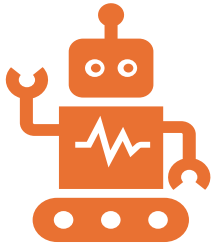
XML

```
<Teacher>
  <Name>Piyush</Name>
  <Age>34</Age>
</Teacher>
```

XAML

```
<StackLayout>
  <Label Text="Piyush" />
  <Label Text="34" />
  <Button Text="Click Me" />
</StackLayout>
```

MVC and MVVM in Xamarin

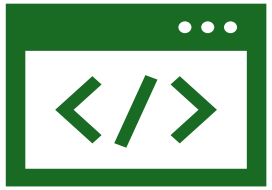


MVC (Model–View–Controller):

Model = data, View = UI, Controller = handles input and updates View.

UI updates are **manual** (controller tells view what to show).

More common in **web apps**.



MVVM (Model–View–ViewModel):

Model = data, View = XAML UI, ViewModel = exposes properties/commands.

UI updates are **automatic** via **data binding**.

Best suited for **Xamarin.Forms apps**.

Key Difference: MVC requires manual UI updates, while MVVM uses binding for automatic updates, making it more testable and maintainable in Xamarin.

Xamarin

- A **cross-platform framework** that lets you build mobile apps for **Android and iOS** using **C# and .NET**.
- You write most of your code once (business logic, backend, models) and share it across platforms.
- But the **UI is written separately** for each platform (Android XML layouts, iOS Storyboards).
- So, you still need to design **two different UIs**.
- Think of Xamarin as: *"Shared logic, separate UIs."*

Xamarin.Forms

- A **UI toolkit built on top of Xamarin**.
- Lets you write **one shared UI in XAML** that runs on **Android, iOS, and UWP**.
- Uses **data binding, layouts, and controls** (like Button, Label, Grid) that automatically render as native controls on each platform.
- Great for apps where you want **maximum code and UI reuse**
- Think of Xamarin.Forms as: *"Shared logic + shared UI."*

Feature	Xamarin	Xamarin.Forms
UI Code	Written separately for Android/iOS	Written once in XAML, shared
Code Sharing	Business logic only	Business logic + UI
Flexibility	More control over native UI	Faster development, less native customization
Best For	Apps needing custom native UI	Apps needing quick cross-platform UI